



Máster en Inteligencia Artificial, Cloud Computing y DevOps

Machine Learning y Deep Learning

Guión entrega final

Sergio Benito Martín

Índice

Índice	3
 Introducción	4
 Objetivo General	4
 Requisitos mínimos	4
 Criterios de evaluación	5
 Entregables obligatorios	6
 Conclusión	7
 Anexo: Criterios de evaluación	8
 1. Implementación y comparación de modelos	8
 2. Evaluación técnica y visualización	8
 3. Automatización y estructura del sistema	9
 4. Calidad del código y documentación	9
 5. Bonus técnicos	9

Introducción

El módulo de Machine Learning y Deep Learning debe **ofrecer un conocimiento base sobre los fundamentos del uso de algoritmos de aprendizaje automático** que permitirán la adquisición de conocimientos más específicos sobre la Inteligencia Artificial Generativa.

Con el objetivo de poder **preparar a los alumnos adecuadamente para adquirir dichos conocimientos y ser capaces de resolver casos de uso reales en su futuro profesional**, la evaluación final constará de un ejercicio a resolver centrado en el uso de algoritmos de aprendizaje automático a la vez que se utilizan herramientas y tecnologías ya vistas en módulos previos, de tal manera que **el aprendizaje se vaya realizando de manera iterativa e incremental**.

Objetivo General

Uno de los grandes avances en la industria ha sido el nacimiento de muchas tecnologías de low-code que permiten entrenar modelos de manera muy rápida y eficiente, probando diferentes algoritmos al mismo tiempo y ofreciendo el resultado al usuario denominadas librerías de AutoML, algunos ejemplos son PyCaret, MLJar, H2O, TPOT, etc. Gracias a estas tecnologías, el proceso de prototipado de un caso de uso es muy rápido e, incluso, de construir una solución útil para el destinatario final.

Para ello, es necesario que dichas librerías estén construídas de manera muy modular y con una orientación a objetos que permita hacer transparente el proceso de entrenamiento y comparación de dichos modelos.

Diseñar e implementar un sistema automático que:

- Entrene, evalúe y compare distintos modelos de clasificación binaria.
- Seleccione el mejor modelo según una métrica principal, además de ofrecer una visión de otras secundarias.
- Automatice el flujo completo desde los datos hasta la inferencia.

Se propone adjunto el dataset que se utilizará para llevar a cabo el proyecto..

Requisitos mínimos

1. **Problema de clasificación binaria**
 - Dataset real proporcionado en el apartado previo, con una variable objetivo binaria (0 o 1).
 - Justificación del problema y del conjunto de datos.

2. Modelos a implementar y comparar

Se deben entrenar al menos los siguientes algoritmos, además de otros que puedan ser de interés:

- Regresión logística
- Árbol de decisión
- Random Forest
- Gradient Boosting (XGBoost, LightGBM o CatBoost)
- Red neuronal multicapa usando Keras de TensorFlow

3. Evaluación de modelos

- Utilizar al menos una de las siguientes métricas como principal: accuracy, precision, recall, F1-score, AUC-ROC.
- Se debe justificar el por qué se ha elegido dicha métrica como principal.
- Mostrar: matriz de confusión y curva ROC.

4. Automatización del flujo

- Implementar un pipeline estructurado para: carga de datos, preprocesamiento, entrenamiento, evaluación, y selección del mejor modelo.



Criterios de evaluación

La práctica **se realizará por parejas**, pudiéndose formar por elección propia. En caso de no encontrar a una persona con quien formar pareja házselo saber al profesor. El objetivo es fomentar y promover una situación real de trabajo similar a la que poderse enfrentar en el futuro, por tanto es necesario encontrar la buena forma de colaborar en equipo.

Al igual que en un caso real, se valora la entrega en su conjunto y por tanto será el peso mayoritario a nivel de evaluación pero, dado que es un contexto académico y formativo, se valorará individualmente la contribución y aprendizaje de cada uno de los integrantes de la pareja. Para poderlo llevar a cabo:

- Se utilizará la actividad de cada integrante mediante el historial de commits para poder entender la contribución y colaboración conjunta.
- Se tendrá en cuenta la autonomía, aportaciones técnicas y visibilidad del trabajo en el repositorio.
- En la documentación habrá que indicar en un apartado los roles llevados a cabo, quién se hace cargo de qué parte.

En caso de que no se haga una distinción específica de los roles de cada una de las personas integrantes de la pareja, se asignará la misma nota a ambas personas.

Además, también con el fin de poder acercar el desarrollo de las prácticas a la realidad laboral como si de un proyecto profesional se tratase, **se deberá hacer una defensa de la práctica con el profesor**.

Dicha defensa constará de un **intercambio de 30 minutos máximo** en los que tras ejecutar el código por parte de los integrantes de la pareja, el profesor efectuará diversas preguntas a los alumnos para comprender diversos aspectos de la implementación. Cuestiones a tener en cuenta:

- El objetivo será **mostrar la ejecución del código y explicar las diversas decisiones tomadas** para haber optado por dicha implementación.
- Será necesaria para poder llevar a cabo la evaluación del proyecto.
- Dicha defensa se realizará a posteriori de la entrega de la práctica.
- La reserva del momento de la defensa se podrá realizar por parte de una de las personas de la pareja a través del [siguiente calendario](#) anotando los nombres y correos electrónicos de ambas. En caso de no encontrar un hueco que pueda convenir a la pareja, hacedlo saber al profesor para encontrar juntos un momento donde fecharla.

Los criterios de evaluación se resumen a continuación adjuntando un mayor detalle en el [Anexo](#) al final del documento:

Criterio	Peso
Implementación y comparación de modelos	30%
Evaluación técnica y visualización	25%
Automatización y estructura del sistema	25%
Calidad del código y documentación	20%
Bonus técnicos (objetos, API, embeddings...)	+20%

Entregables obligatorios

Cada pareja deberá entregar el siguiente material para la correcta evaluación del proyecto así como la documentación correspondiente:

-  **Repositorio de código (GitHub):**
 - Código bien estructurado, separando los diferentes scripts, notebooks y distintos archivos.
 - Documentación completa en README con:
 - Autores de la práctica
 - Descripción del problema y datos
 - Instrucciones para ejecutar el proyecto
 - Resultados y conclusiones
 - Archivos auxiliares:

- `.gitignore` adecuado
- `requirements.txt` con todas las dependencias
-  **Informe final en PDF o Markdown:**
 - Definición de los roles de la pareja (quién hace qué)
 - Justificación del problema
 - Análisis exploratorio de datos
 - Diseño del sistema
 - Resultados y elección final
 - Reflexión crítica sobre limitaciones y mejoras

El código entregado se debe poder ejecutar sin errores para poder asegurar una evaluación correcta, por tanto es importante garantizar que tanto la versión de Python y las librerías utilizadas están incluidas correctamente dentro del README.md y los `requirements.txt`, indicando dentro de dicho README los pasos para poder configurar el entorno virtual y ejecutar el código.

Las parejas se inscribirán en un archivo facilitado por el profesor y la entrega se realizará a través de la plataforma de PontIA, donde cada persona de la pareja subirá el PDF y que debe ser igual para ambas además de contener la URL del repositorio de Github donde está construída la resolución del problema.

Conclusión

Esta práctica permite entender el fundamento de la Inteligencia Artificial, cómo funcionan los algoritmos típicos dentro de esta área, para de esta manera sentar una buena base a la hora de comprender y diseñar sistemas con estas tecnologías.

Es un inicio de lo que sería un proyecto puramente real, al que en un futuro muy cercano los alumnos se enfrenten tras ocupar posiciones dentro de la industria. Por tanto la manera de llevarlo a cabo tiene que ser lo más fiel a dicho momento, al trabajar con otras personas en un mismo objetivo, al investigar tecnologías comunes en el sector para finalmente proveer de una solución adecuada a la necesidad de negocio planteada.



Anexo: Criterios de evaluación



1. Implementación y comparación de modelos

Qué se evalúa:

- Que el alumno implemente correctamente al menos 5 modelos distintos (regresión logística, árbol, bosques aleatorios, boosting, red neuronal).
- Que entrene los modelos y los compare usando un enfoque metodológico coherente.

Ejemplos valorados:

- Comparar modelos usando un conjunto de métricas comunes.
- Justificar por qué un modelo supera a otro en un caso concreto.
- Mostrar resultados en una tabla similar a la siguiente:

Modelo	Accuracy	F1-score	ROC-AUC
Logistic Regression	0,88	0,85	0,91
Decision Tree	0,89	0,83	0,92
Random Forest	0,91	0,89	0,94
XGBoost	0,92	0,90	0,95
Deep Neural Network (Keras)	0,89	0,87	0,93



2. Evaluación técnica y visualización

Qué se evalúa:

- Aplicación de métricas de evaluación relevantes según el problema (precisión, recall, AUC...).
- Inclusión de visualizaciones como curvas ROC, matrices de confusión, gráficos de importancia de variables.
- Análisis Exploratorio de Datos que ha servido para tomar las diferentes decisiones en cuanto al implementación de los modelos.

Ejemplos valorados:

- Curva ROC comparativa entre modelos.
- Matriz de confusión con etiquetas legibles.
- Gráfico de `feature_importances_` de un Random Forest.

3. Automatización y estructura del sistema

Qué se evalúa:

- Que el proceso esté empaquetado en un pipeline claro (no todo dentro de un notebook suelto).
- Separación de funcionalidades en distintos módulos o scripts (`data_loader.py`, `trainer.py`, etc.).

Ejemplos valorados:

- Script principal (`trainer.py`) que lanza todo el flujo de trabajo.
- Uso de funciones o clases reutilizables.
- Pipeline de Scikit-learn bien estructurado.

4. Calidad del código y documentación

Qué se evalúa:

- Uso de buenas prácticas de codificación (PEP8, nombres descriptivos, comentarios).
- Documentación adecuada en el README y dentro del código (docstrings).

Ejemplos valorados:

- Clases con métodos documentados.
- README claro con instrucciones de uso y estructura del proyecto.
- `.gitignore` y organización por carpetas.

5. Bonus técnicos

Gracias a una buena implementación de cualquiera de los siguientes puntos se puede alcanzar **hasta 2 puntos adicionales sobre la nota calculada** con el resto de [requerimientos mínimos obligatorios](#), permitiendo de esta manera alcanzar un 10. Es **necesario que el sistema funcione para poder sumar la puntuación adicional**.

Bonus técnico	Descripción
 API con FastAPI	Crear una API REST para exponer endpoints como <code>/train</code> , <code>/predict</code> o <code>/evaluate</code> .
 Embeddings personalizados	Uso de Word2Vec, TF-IDF o embeddings categóricos para representar variables complejas.
 Optimización de hiperparámetros	Aplicación de <code>GridSearchCV</code> o <code>RandomizedSearchCV</code>
 Registro de experimentos	Utilizar <code>MLFlow</code> para registrar los experimentos realizados con los diferentes algoritmos y registrar el modelo final
 Balanceo de clases	Aplicación de técnicas como <code>SMOTE</code> , <code>undersampling</code> , <code>class_weight</code> .
 Interpretabilidad	Uso de <code>SHAP</code> , <code>LIME</code> , o <code>feature_importances_</code> .
 Interfaz visual	Presentación vía <code>Streamlit</code> , <code>Gradio</code> o web UI simple.

! FAQs: Preguntas y respuestas

¿Hay que entregarla en un notebook?

La idea es que creéis un proyecto de software, con sus diferentes módulos, siguiendo los principios que habéis visto en asignaturas previas. Esta parte es clave, porque tendréis que crear funciones, clases (si lo consideráis necesario) y una estructura que permita ejecutar el código como si de un script se tratase

Los notebooks serán necesarios, por supuesto, pero para que podáis hacer el análisis exploratorio de datos (EDA) o pruebas con los diferentes algoritmos, también añadirlos dentro del repositorio en una carpeta que se puede llamar "notebooks/".

¿Es necesario crear flujos de CI/CD?

No sería necesario.

El objetivo de la práctica es productivizar el entrenamiento y evaluación de diferentes modelos para resolver un problema de clasificación binaria. La clave es que os podáis centrar en esa parte más de trabajar con los datos y los algoritmos a la vez que lo productivizáis.

Sin embargo todo lo que podáis realizar a mayores y que enriquezca los requisitos mínimos lo valoraré positivamente como un bonus y os permitirá aumentar la nota adicional. Respecto a dichos aspectos bonus os he dado algunos ejemplos en la definición de la práctica.

¿Existe una estructura de repositorio que haya que seguir?

Se recomienda una similar a la siguiente pero el formato es libre mientras se separen los notebooks de exploración del código fuente necesario para ejecutar el pipeline de entrenamiento y/o evaluación. También se debe documentar el README.md y si fuera necesario complementar con otra documentación adicional.

```

proyecto-final-ml/
├── .gitignore                                # 🗑️ Archivos que no se suben al repo (e.g., modelos, datos temporales)
├── data/                                     # 📁 Datos usados en el proyecto
│   ├── raw/                                # Datos originales sin procesar (por ejemplo, CSV descargado o incluso Parquet)
│   └── processed/                          # Datos tras limpieza y transformación (listos para modelar)
├── docs/                                    # 📖 Documentación adicional
├── models/                                 # 📁 Modelos entrenados (guardados con joblib o pickle)
│   ├── tests/                             # (opcional) Modelos intermedios de prueba
│   │   ├── logistic_regression.pkl         # Ejemplo: Modelo de Regresión Logística
│   │   ├── tree.pkl                       # Ejemplo: Modelo de Árbol de Decisión
│   │   ├── random_forest.pkl             # Ejemplo: Modelo Random Forest
│   │   ├── xgboost.pkl                   # Ejemplo: Modelo XGBoost
│   │   ├── lightgbm.pkl                  # Ejemplo: Modelo LightGBM
│   │   ├── neural_network.pkl            # Ejemplo: Red Neuronal
│   └── best_model.pkl                    # El mejor modelo seleccionado para producción
├── notebooks/                             # 📁 Todos los notebooks del proyecto
│   ├── exploracion/                      # Notebooks de pruebas, EDA inicial, prototipos
│   │   ├── eda_inicial.ipynb             # Ejemplo: análisis inicial del dataset
│   │   └── pruebas_modelos.ipynb         # Ejemplo: pruebas de diferentes modelos
│   └── finales/                          # Notebooks finales con resultados o presentación
│       ├── eda_final.ipynb               # Ejemplo: EDA del dataset
│       └── comparativa_modelos.ipynb     # Ejemplo: comparación final de modelo usada para los scripts
├── outputs/                               # 📁 Gráficos, reportes y resultados generados (PNGs, HTMLs si son interactivos, etc)
│   ├── confusion_matrix.png
│   └── feature_importance.html
├── src/                                   # 🧠 Código fuente del proyecto
│   ├── __init__.py                       # Inicializador del paquete src
│   ├── config.py                         # Parámetros y configuración del proyecto
│   ├── data_loader.py                   # Funciones para cargar y transformar datos
│   ├── model_trainer.py                 # Clase o funciones para entrenar modelos
│   ├── evaluator.py                     # Métricas y visualización de resultados
│   └── predictor.py                     # Funciones para hacer predicciones con modelos entrenados
├── requirements.txt                       # 📦 Dependencias del proyecto
└── README.md                             # 📖 Documentación principal del proyecto con comandos de ejecución
    
```

¿La documentación es el README.md?

Si se incluye toda la información necesaria definida en el apartado de [📁 Entregables obligatorios](#), sin problema, pero deben estar todos los puntos recogidos.